In Java 8, these four items are called **Functional Interfaces**. They are like templates for short, quick blocks of code (Lambda expressions) that you can plug into your Streams to transform, filter, or create data.
The easiest way to understand the difference is to look at **what goes in** (Inputs) and **what comes out** (Outputs).

Quick Comparison Table

Interface	Inputs	Outputs	Everyday Analogy	Common Stream Use
`Function`	1 Input	1 Output	A factory machine changing steel into a car.	`.map()`
`BiFunction`	2 Inputs	1 Output	A chef mixing flour and water to make dough.	`.reduce()`
`Predicate`	1 Input	`boolean` (True/False)	A security guard checking IDs at a door.	`.filter()`
`Supplier`	0 Inputs	1 Output	A vending machine dropping snacks from nowhere.	`.collect()`
`Consumer`	**1 Input**	**`void` (Nothing)**	**A trash can eating garbage and keeping it.**	**`.forEach()`**
`BiConsumer`	**2 Inputs**	**`void` (Nothing)**	**A mailman putting a letter inside a specific mailbox.**	**`.collect()` (into a Map)**

---

1. Function<T, R>

- **What it does:** It takes **one** thing, changes it, and gives you **one** new thing back.
- **Stream Example:** Changing a list of lowercase names into UPPERCASE names using `.map()`.

```
// Takes a String, returns the length of that String (an Integer)
Function<String, Integer> stringLength = str -> str.length();

List<Integer> lengths = names.stream()
                             .map(stringLength) // Changes "Java" into 4
                             .collect(Collectors.toList());
```

2. BiFunction<T, U, R>

- **What it does:** The "Bi" means two. It takes **two** separate inputs, combines or processes them, and gives you **one**result back.
- **Stream Example:** Adding numbers together inside a `.reduce()` step.

```
// Takes two Integers, adds them together, and returns the result
BiFunction<Integer, Integer, Integer> addNumbers = (num1, num2) -> num1 + num2;
```

(Note: Because Streams process items one by one, you usually see specialized versions of this in streams, like `BinaryOperator`.)

3. Predicate

- **What it does:** It takes **one** input, tests it against a condition, and answers either **true** or **false**.
- **Stream Example:** Throwing away numbers that are too small using `.filter()`.

```
// Takes an Integer, checks if it is bigger than 10
Predicate<Integer> isBigNumber = num -> num > 10;

List<Integer> bigNumbers = numbers.stream()
                                  .filter(isBigNumber) // Only keeps true answers
                                  .collect(Collectors.toList());
```

4. Supplier

- **What it does:** It takes **zero** inputs, but it hands you a brand-new object every time you call it. It "supplies" data.
- **Stream Example:** Telling a stream how to build a brand new container (like an empty `ArrayList`) to hold your results at the end.

```
// Takes nothing, but creates a brand new ArrayList whenever asked
Supplier<List<String>> listSupplier = () -> new ArrayList<>();

List<String> result = names.stream()
                           .collect(Collectors.toCollection(listSupplier));
```

5. Consumer

- **What it does:** It takes **one** input, does a job with it (like printing or saving), and gives absolutely nothing back.
- **Stream Example:** Printing every item in a list at the very end of your stream using `.forEach()`.

```
// Takes a String and prints it to the screen
Consumer<String> printName = name -> System.out.println(name);

names.stream()
     .filter(name -> name.startsWith("J"))
     .forEach(printName); // Eats each name and prints it
```

6. BiConsumer<T, U>

- **What it does:** It takes **two** separate inputs, does a job with them, and gives nothing back.
- **Stream Example:** Pulling keys and values out of a `Map` to display them, or building a custom `Map` using `.collect()`.

```
// Takes two inputs (a Key and a Value) and prints them together
BiConsumer<String, Integer> printScores = (name, score) ->
    System.out.println(name + " scored " + score);

myMap.forEach(printScores); // Processes both items at once
```